

Exercise 1: Properties, Contracts, and Specification Automata

1 Property Types

```
1 struct Item {
2     int priceInCent;
3     struct Item *next;
4 };
5
6 int read_discount(void);
7
8 int order_total(struct Item *item, int count) {
9     int discount = read_discount();
10    int total = 0;
11    for (int i = 0; i < count; i++) {
12        total+=item->priceInCent;
13        item = item->next;
14    }
15    return total * (100 - discount) / 100;
16 }
```

- a. Classify each property as (a) reachability, (b) liveness, (c) performance, or (d) code quality. State a reason for each classification.

1. φ_0 = No signed integer overflow in the sum or product.

***Pre-filled solution:** Reachability. The property is violated as soon as there is an overflow reachable on any path.*

2. φ_1 = Every value returned by `read_discount` lies between 0 and 100.
3. φ_2 = Every started call to `order_total` eventually returns.
4. φ_3 = Every call completes within 5 ms on the specified hardware.
5. φ_4 = Every `+=` operator has a space on each side.

- b. Propose one additional property for the above program. Classify it and state whether it is fulfilled or violated.

2 Contracts

Consider the below function and its specified contract.

```
1 int apply_discount(int totalInCent, int discount) {  
2     int discounted = totalInCent * (100 - discount) / 100;  
3     return discounted;  
4 }
```

Contract.

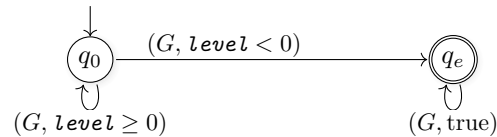
1. Preconditions: $0 \leq total \leq 10,000$ and $0 \leq discount \leq 100$.
 2. Postcondition: $0 \leq discounted \leq total$.
- c. Write a version of the above function that includes `assert` checks for this contract. Do these checks ensure that the discount amount is correct?

3 Specification Automata

For each property, **draw** a **complete** specification automaton. Assume that set G is the set of all program statements.

- (a) The variable `level` is always greater or equal to 0.

Pre-filled solution: The complete automaton has initial state q_0 and accepting state q_e .



q_0 means that no negative value has been observed. q_e records a violation, even if `level` becomes greater or equal to 0 again.

- (b) The function `reach_error()` is never called.

- (b) Always `div` $\neq$ 0.

- (c) Whenever `step` $>$ 2, `step` must stay $>$ 2 until `step` = 0.
`step` may stay $>$ 2 forever; it does not have to become 0.